

Ri-acquisizione del corpus Dataverse — Runbook Fase 2 (cronaca dell'esecuzione)

Direzione ICT, Università degli Studi di Milano — Progetto Arkive

Luglio 2026

Indice

Runbook — Fase 2 (cronaca dell'esecuzione)	1
16. Sintesi dell'esecuzione	1
17. Revisione della politica sugli archivi compressi	2
18. Approvazione automatica dei transfer sotto carico	2
19. Stato inconsistente: transfer, SIP, task e job “fantasma”	3
19.1 Doppioni da avvii paralleli	4
20. La causa di fondo: l'orologio di WSL	4
21. Robustezza acquisita dalla toolchain	5
22. Gate finale e pulizia del corpus	5
22.1 Doppioni residui	5
22.2 Procedura di rimozione sicura	6
22.3 Reindicizzazione finale	6
23. Raccomandazioni infrastrutturali	6
24. Stato alla chiusura della Fase 2	7

Runbook — Fase 2 (cronaca dell'esecuzione)

Questo documento è la continuazione operativa del *Runbook del pilota (Fase 1)* e della sezione 15 di quel documento, che ne descrive l'**impostazione** e il **piano** di migrazione. Qui si registra invece ciò che è **realmente accaduto** durante l'ingest di produzione dei 27 lotti: le decisioni prese in corso d'opera, i blocchi incontrati e i rimedi adottati. La numerazione delle sezioni prosegue idealmente quella del runbook di Fase 1 (che si chiude alla sezione 15).

16. Sintesi dell'esecuzione

La ri-acquisizione di produzione ha portato in conservazione **671 DOI** — l'intero scope raggiungibile (708 DOI totali, meno 30 vuoti e 7 non più raggiungibili, sezione 8) — organizzati in 27 lotti bilanciati sul numero di file (67.730 file, ~63 GB).

L'esecuzione si è svolta in più sessioni sull'arco di alcuni giorni. Non è stata lineare: una catena di problemi **sovrapposti** — alcuni dei dati, molti dell'ambiente WSL — ha richiesto diagnosi ripetute prima di isolare le cause vere. Ogni causa, una volta individuata, è stata risolta alla radice; il risultato è un corpus completo e verificato, e una toolchain più robusta di quella iniziale.

Le cause di fondo, in ordine di impatto, sono state:

1. **l'orologio di sistema di WSL** che saltava all'indietro, causando fallimenti a cascata nei job di storage (sezione 20) — il fattore che rendeva il comportamento imprevedibile;

2. l'**estrazione degli archivi compressi**, che moltiplicava i file da caratterizzare e saturava la coda (sezione 17);
3. l'**approvazione automatica dei transfer** che, sotto carico, lasciava pacchetti in attesa senza avanzare (sezione 18);
4. lo **stato inconsistente** lasciato da interruzioni e riavvii — task e job “fantasma” che bloccavano la coda (sezione 19).

Le sezioni seguenti trattano ciascuna causa come caso operativo autonomo, sul modello della sezione 9 del runbook di Fase 1.

17. Revisione della politica sugli archivi compressi

Contesto. La sezione 10 (Fase 1) aveva adottato l'estrazione degli archivi (*Extract packages = Yes, Delete after extraction = No*): l'AIP conteneva sia l'archivio originale sia il contenuto estratto, identificato e normalizzato. Su un pilota ristretto era la scelta più conservativa.

Il problema su scala reale. Su 27 lotti la scelta si è rivelata insostenibile. Quando Archivematica estrae un archivio, **ogni file interno viene caratterizzato con FITS** (~5 secondi per file su questa installazione). Un dataset contenente archivi con migliaia di file interni esplode così in migliaia di task di caratterizzazione: la coda si satura, la lavorazione rallenta fino a fermarsi, e in alcuni casi il transfer fallisce del tutto.

Il caso emblematico è stato **LJ6Z8V** (12 archivi .zip contenenti modelli ML): con l'estrazione attiva il transfer finiva sistematicamente in failed.

Decisione adottata — conservazione *as-is*. Si è **disabilitata l'estrazione di tutti i formati-contenitore** (ZIP, 7z, TAR, RAR, CAB, BZip), lasciando attive solo le regole per le immagini-disco (ISO, AFF, EO1...), assenti dal corpus. Gli archivi vengono **identificati** (PUID corretto) ma **conservati interi** nell'AIP, senza estrazione.

Questa scelta ribalta quella di Fase 1, e la motivazione è cambiata con la scala e con la natura del corpus:

- il corpus è collegato a un Dataverse istituzionale, dove l'oggetto originale resta comunque disponibile: l'AIP è la copia di **conservazione**, e conservare l'archivio *come depositato* è la scelta più fedele;
- è coerente con la politica già adottata per il GZip (sezione 10.4): l'archivio si conserva intero, identificato ma non estratto;
- elimina in un colpo sia i fallimenti sia la saturazione della coda.

Il prezzo — il contenuto interno agli archivi non è indicizzato né caratterizzato singolarmente — è accettabile per un archivio di conservazione con l'originale sempre reperibile a monte.

```
-- Disabilitazione delle regole di estrazione dei formati-contenitore.
-- (prova in transazione, poi COMMIT)
UPDATE fpr_fprule SET enabled = 0
WHERE purpose = 'extract' AND enabled = 1
  AND command_id IN (
    SELECT uuid FROM fpr_fpcmd
    WHERE description LIKE '%7Zip%' OR description LIKE '%RAR%'
  );
```

Conseguenza sulla pianificazione dei lotti. Con l'estrazione disabilitata, i dataset con archivi non richiedono più un trattamento separato: rientrano nei lotti ordinari e si conservano *as-is*. La “passata dedicata GZip” prevista in Fase 1 diventa superflua.

18. Approvazione automatica dei transfer sotto carico

Sintomo. La lavorazione avanza per un po', poi sembra fermarsi: `ultimo_job` nel database resta indietro di ore, `in_corso` (task attivi) è zero, ma sul disco e nella coda si accumulano transfer che non progrediscono. Interrogando l'API si trovano diversi transfer in stato **unapproved**.

Causa. L’approvazione automatica di `archivematica_ingest.py` è per-DOI: dopo aver avviato un transfer lo cerca fra gli `unapproved` e lo approva. Nei lotti “leggeri” (centinaia di DOI piccoli avviati in rapida successione), o quando la coda è temporaneamente rallentata, una parte delle approvazioni **non va a buon fine** e i transfer restano in attesa. Un transfer non approvato non genera task: la coda *sembra* bloccata mentre in realtà **aspetta approvazioni** che non arrivano.

La riprova è netta: approvando **a mano** i transfer in attesa, questi vengono processati fino all’AIP senza altri interventi. La lavorazione funziona; è l’approvazione l’anello debole.

Rimedio — watchdog di approvazione. Si è introdotto uno strumento dedicato, `approva_watchdog.py`, che gira **in parallelo** alla sequenza di ingest e ogni 30 secondi approva *tutto* ciò che trova in attesa. L’approvazione non dipende più dal singolo `archivematica_ingest.py`: qualunque transfer compaia viene preso in carico entro mezzo minuto.

```
# il watchdog va lanciato in modo che sopravviva alla sessione: setsid, non nohup
setsid python3 approva_watchdog.py > logs/watchdog_$(date +%Y%m%d_%H%M).log 2>&1 &
```

Il watchdog deve restare vivo per tutta la sequenza. Se termina (chiusura della sessione, riavvio), i transfer tornano ad accumularsi. Verificare periodicamente che il processo sia attivo: `ps aux | grep '[a]pprova_watchdog'` deve dare una riga.

19. Stato inconsistente: transfer, SIP, task e job “fantasma”

Sintomo. Dopo un’interruzione (riavvio della macchina, kill di uno script, timeout), Archivematica risulta con tutti i servizi active ma **non processa nulla**: nessun job nuovo, nessun task in esecuzione, e la coda ferma.

Causa. Un’unità (transfer o SIP) interrotta a metà lascia nel database **task con endTime IS NULL** (avviati e mai conclusi) e **job a currentStep = 3** (in esecuzione). Archivematica li considera ancora attivi e non procede, in attesa del loro completamento — che non arriverà mai, perché il processo che li eseguiva è morto. Sono record “fantasma”.

Il MCPServer, inoltre, **ricrea i job dalle watched directory a ogni riavvio**: un pacchetto rimasto fisicamente in `currentlyProcessing` o in `activeTransfers` viene ripreso, con il rischio di generare un **doppione** se il suo AIP era già stato archiviato prima dell’interruzione.

Rimedio — chiusura dei fantasmi. A servizi fermi, si chiudono i task e i job appesi dell’unità morta, portandoli a uno stato terminale, e si rimuovono le copie fisiche residue prima di riavviare.

```
-- 1. chiudere i task appesi (endTime nullo) dell'unità
UPDATE Tasks t JOIN Jobs j ON t.jobuuid = j.jobUUID
SET t.endTime = NOW(), t.exitCode = 1,
    t.stdError = 'Chiuso: task fantasma da interruzione'
WHERE j.SIPUUID LIKE '<uuid8>%' AND t.endTime IS NULL;

-- 2. portare i job in esecuzione (step 3) o in attesa (step 1) a stato
--    terminale (step 4 = failed)
UPDATE Jobs SET currentStep = 4
WHERE SIPUUID LIKE '<uuid8>%' AND currentStep IN (1, 3);

# 3. rimuovere le copie fisiche residue PRIMA di riavviare MCPServer
sudo systemctl stop archivematica-mcp-server
for base in currentlyProcessing failed activeTransfers/standardTransfer; do
    sudo find /var/archivematica/sharedDirectory/$base -maxdepth 1 \
        -name "*<DOI>*" -exec rm -rf {} + 2>/dev/null
done
sudo systemctl start archivematica-mcp-server
sudo systemctl restart archivematica-mcp-client # sempre, dopo il server (sez. 9.2)
```

Riconoscere i fantasmi dai numeri. Il segnale non è lo stato dei servizi (che restano *active*), ma il fatto che `SELECT MAX(createdTime) FROM Jobs` resti indietro nel tempo mentre `SELECT COUNT(*) FROM Tasks WHERE endTime IS NULL` è maggiore di zero. Task iniziati ore prima e mai conclusi occupano la coda a vuoto.

19.1 Doppioni da avvii paralleli

Un caso particolare di inconsistenza nasce dal **doppio lancio** della sequenza: due istanze che processano gli stessi lotti in parallelo producono un AIP doppio per ogni DOI. È accaduto per NZRX1C (due AIP identici a sette minuti di distanza).

Rimedio — **guardia anti-doppio-lancio.** `ingest_sequenza.sh` acquisisce all'avvio un *lock file*; una seconda istanza che trovi il lock di un processo ancora vivo si rifiuta di partire, e rimuove invece un lock orfano (processo non più attivo).

```
LOCK="/tmp/arkive_ingest_sequenza.lock"
if [ -e "$LOCK" ]; then
    pid=$(cat "$LOCK" 2>/dev/null)
    if [ -n "$pid" ] && kill -0 "$pid" 2>/dev/null; then
        echo "ERROR: un'altra sequenza è già in esecuzione (PID $pid)."; exit 1
    fi
    rm -f "$LOCK"          # lock orfano
fi
echo $$ > "$LOCK"; trap 'rm -f "$LOCK"' EXIT
```

20. La causa di fondo: l'orologio di WSL

Sintomo. Job di *Store AIP* che falliscono con un errore inatteso:

```
ValueError: Counters can only be incremented by non-negative amounts.
... ss_api_time_counter.labels(**kwargs).inc(duration)
```

L'AIP non viene scritto; la coda si blocca a partire dal fallimento.

Causa. L'errore nasce da un contatore di metriche (Prometheus) che misura la **durata** di una chiamata all'API dello Storage Service: `durata = fine - inizio`. Se l'orologio di sistema **salta all'indietro** fra l'inizio e la fine della misura, la durata risulta **negativa** e il contatore, che accetta solo incrementi non-negativi, solleva l'eccezione — facendo fallire l'intero job.

La causa dei salti è un **conflitto di sincronizzazione dell'orologio in WSL**: il servizio `systemd-timesyncd` interno alla distribuzione tenta di sincronizzare l'ora in concorrenza con l'orologio ereditato da Windows. Il risultato, nei log di sistema, è un `Clock change detected` **ogni ~30 secondi** e occasionali `Time jumped backwards`.

Questa instabilità è il fattore che rendeva **imprevedibile** l'intera lavorazione: non solo i fallimenti di *Store AIP*, ma verosimilmente anche parte dei comportamenti anomali della coda e dei timer.

Rimedio — **disabilitare la sincronizzazione interna a WSL.** Poiché l'ora ereditata da Windows è corretta, la sincronizzazione NTP interna è superflua e dannosa. Va disattivata e resa persistente.

```
# fermare i salti (verifica: l'ora resta corretta perché la fornisce Windows)
sudo timedatectl set-ntp false

# controllo: nei successivi 90 s non devono più comparire salti
sudo journalctl --since "90 seconds ago" | grep -icE "clock change|time jump" # atteso: 0

# rendere permanente (altrimenti si riattiva al riavvio di WSL)
sudo systemctl disable systemd-timesyncd
```

Diagnosi rapida. Il sintomo dei salti è visibile prima ancora che un job fallisca: `sudo journalctl --since "5 min ago" | grep -iE "clock change|time jump"`. Righe ripetute a cadenza regolare indicano il conflitto. È un controllo da fare **all’inizio** di ogni sessione di lavorazione su WSL.

21. Robustezza acquisita dalla toolchain

I rimedi delle sezioni precedenti sono stati consolidati negli strumenti, così da non doverli riapplicare a mano:

Strumento	Miglioramento
approva_watchdog.py	nuovo: approvazione continua dei transfer in attesa (sez. 18)
ingest_sequenza.sh	guardia anti-doppio-lancio (sez. 19.1); guardia “coperti < attesi” che distingue gli errori veri dai DOI già presenti e non si ferma sui lotti già completi; timeout di polling elevato (2 ore) per i dataset lenti
riconcilia_stato.py	default del file di stato corretto in <code>stato_riacquisizione.json</code> ; riconcilia le voci <code>failed</code> il cui AIP è comunque presente nello Storage Service
FPR	estrazione dei formati-contenitore disabilitata (sez. 17)
ambiente WSL	<code>systemd-timesyncd</code> disabilitato (sez. 20)

Nota sulla persistenza dei processi in WSL. I processi di lunga durata (sequenza e watchdog) vanno avviati con `setsid`, non con `nohup`: in WSL la chiusura della sessione può terminare comunque i processi lanciati con `nohup`.

22. Gate finale e pulizia del corpus

Completati gli ingest, prima della migrazione si esegue il **Gate finale**: allineamento fra il file di stato, i file su disco e i record nello Storage Service.

```
# 1. file di stato: tutti ingested
python3 -c "import json; from collections import Counter; \
    print(Counter(v['status'] for v in json.load(open('stato_riacquisizione.json')).values()))"

# 2. AIP su disco == AIP nel DB
sudo find /mnt/e/ARKIVE/AIPsStore -name '*.tar.gz' -type f | wc -l
sudo mysql SS -e "SELECT status, COUNT(*) FROM locations_package GROUP BY status;"

# 3. DOI distinti con AIP (deve valere 671)
sudo find /mnt/e/ARKIVE/AIPsStore -name '*.tar.gz' -type f \
    | grep -oE 'RD_UNIMI_[A-Z0-9]+' | sort -u | wc -l
```

22.1 Doppioni residui

Le interruzioni e i riavvii avevano prodotto **14 AIP dopponi** (stesso DOI ingerito due volte). Vanno riconosciuti e rimossi, tenendo per ciascun DOI l’AIP a cui punta il file di stato.

I dopponi hanno dimensioni *quasi* identiche ma non uguali: la differenza (pochi KB su file da centinaia di MB) è dovuta ai **metadati di processo** rigenerati a ogni ingest — UUID dei file, timestamp di elaborazione, cartella `submissionDocumentation` con l’UUID del transfer. Il **contenuto reale** (i file in `data/objects/`, a parità di conteggio) è identico. La verifica va fatta normalizzando via UUID e timestamp dai nomi prima del confronto, non sulle dimensioni.

22.2 Procedura di rimozione sicura

La rimozione tocca `locations_package` e le sue tabelle collegate, e i file fisici. Si procede con **backup**, **controllo di sicurezza**, **prova in transazione**, poi **COMMIT** — mai una cancellazione diretta.

```
# controllo di sicurezza: nessun UUID da scartare deve essere un AIP 'buono'
# (registrato nel file di stato). Se l'intersezione non è vuota: ABORT.

# backup delle tabelle prima di cancellare
sudo mysqldump --single-transaction SS \
    locations_package locations_event locations_file \
    > ~/arkive_backup_${date +%Y%m%d}/SS_prepulizia_doppioni_${date +%H%M}.sql

-- rimozione dal DB (prova con ROLLBACK, poi ripetere con COMMIT)
START TRANSACTION;
SET FOREIGN_KEY_CHECKS=0;
DELETE FROM locations_event WHERE package_id IN (<uuid,...>);
DELETE FROM locations_file WHERE package_id IN (<uuid,...>);
DELETE FROM locations_package WHERE uuid IN (<uuid,...>);
SELECT ROW_COUNT() AS package_rimossi;
SELECT COUNT(*) AS package_rimasti FROM locations_package; -- atteso: 671
SET FOREIGN_KEY_CHECKS=1;
ROLLBACK; -- diventa COMMIT dopo la verifica

# rimozione dei file fisici, poi delle cartelle-quadrante rimaste vuote
sudo xargs -d '\n' rm -f < /tmp/file_da_rimuovere.txt
sudo find /mnt/e/ARKIVE/AIPsStore -type d -empty -delete
```

Il DEL_REQ. L'AIP di test in stato `DEL_REQ` (residuo della calibrazione del FPR) viene rimosso nella stessa passata: è un record da eliminare da `locations_package`, e il suo file fisico era già assente.

Esito. Dopo la pulizia: **671 AIP** su disco, **671 record UPLOADED** nel database, **0 DEL_REQ** — corpus completo e allineato.

22.3 Reindicizzazione finale

Durante gli ingest l'indicizzazione Elasticsearch è disattivata (`SEARCH_ENABLED=false`), quindi a fine lavorazione gli indici sono fermi a uno stato vecchio. Si ricostruiscono in un'unica passata dai file su disco.

```
sudo -u archivematica bash -c ' \
    set -a -e
    source /etc/default/archivematica-dashboard
    /usr/share/archivematica/virtualenvs/archivematica/bin/python \
        -m archivematica.dashboard.manage \
        rebuild_elasticsearch_aip_index_from_files /mnt/e/ARKIVE/AIPsStore --delete-all
'
```

Il path va adattato. Lo script di reindicizzazione ereditato punta a `/var/archivematica/sharedDirectory/www/AIPsStore` (vuoto in questa configurazione): va corretto in `/mnt/e/ARKIVE/AIPsStore`, dove risiedono davvero gli AIP. A reindicizzazione conclusa, l'indice `aips` deve contare 671 documenti.

23. Raccomandazioni infrastrutturali

L'esecuzione ha reso evidenti due limiti dell'ambiente, da portare all'attenzione dei responsabili dell'infrastruttura (Giorgio, Matteo) per la messa in produzione:

1. **L'orologio di WSL è instabile** (sezione 20). La disabilitazione di `systemd-timesyncd` è un rimedio efficace ma è una mitigazione: un ambiente di produzione non dovrebbe dipendere da essa. È un argomento a favore di un ambiente **non-WSL** (macchina o VM Linux nativa) per il servizio in esercizio.

2. **La Transfer Source su Dropbox/drvfs** resta una fonte di attrito (lock transitori, latenza di copia, filesystem case-insensitive — sezioni 15.3 e 6.4bis.1). Anche a prescindere da Dropbox, un filesystem **ext4 nativo e case-sensitive** è la collocazione appropriata per un archivio di conservazione.

Entrambe le raccomandazioni convergono su un'unica indicazione: **l'ambiente di produzione dovrebbe essere Linux nativo su filesystem case-sensitive**, non WSL su drvfs. La ri-acquisizione è stata portata a termine sull'ambiente esistente, ma la sua fragilità ha inciso in modo significativo sui tempi.

24. Stato alla chiusura della Fase 2

- **671 DOI** ri-acquisiti e conservati; corpus allineato (disco = DB = file di stato), doppioni e residui rimossi, indice Elasticsearch ricostruito.
- **7 DOI** non più raggiungibili, da segnalare a Data@UNIMI (sezione 8).
- **Prossimo passo:** migrazione degli AIP in produzione secondo la procedura già definita nella sezione 15.5 (test su un AIP, poi migrazione in blocco con `bulk_register_aips.py`), escludendo gli UUID annotati in `DOPPIONI_DA_ESCLUDERE.txt`.